Facial Feature Classification Challenge

Cat and Dog Facial-Feature Recognition with Transfer Learning and Ensembling

Auckland University of Technology

Author: Suemon Kwok

Student ID: 14883335

Kaggle name: Suemon Kwok

Competition: ENGE707-2026-S2: Facial Feature Classification Challenge

GitHub: https://github.com/Suemon-Kwok/Kaggle-Competition-Facial-Feature-Classification-Challenge

Contents

Summary 3
Task 1: Dataset Exploration and Understanding 4
1.1 Classification Problem and Dataset 4
1.2 Class Distribution and Balance 4
1.3 Representative Sample Images 5
1.4 Data Quality Assessment 6
1.5 Dataset Usage Strategy 10
Task 2: Data Preprocessing and Pipeline Preparation 11
2.1 Label Encoding and Train/Validation Split 11
2.2 Resizing, Normalisation, Data Loading and Consistent Test Pipeline 12
2.3 Data Augmentation and Leakage Check 14
Task 3: Model Selection and Pipeline Development 16
3.1 Choice of architecture and justification 16
3.2 Architecture 16
3.3 Loss, Optimiser and Hyperparameters 17
3.4 Implementation and Reproducibility 17
Task 4: Training Strategy and Overfitting/Underfitting Management 18
4.1 Two-stage transfer learning 18
4.2 Stage 1: frozen backbone (mild underfitting) 18
4.3 Stage 2: fine-tuning and the onset of overfitting 18
4.4 ResNet50V2: more capacity, more overfitting 19
4.5 Regularisation experiment: weight decay and label smoothing 20
4.6 Summary of strategy changes and their justification 24
Task 5: Model Evaluation and Kaggle Submission 24
5.1 Validation Results 24
5.2 Kaggle Submission and Leaderboard Evidence 26
5.3 Comparing Validation and Leaderboard Performance 27
Task 6: Results Analysis and Improvement Strategies 28
6.1 Where the errors are 28
6.2 Error rate and image resolution 28
6.3 Comparison of experiments and effectiveness of design choices 29
6.4 Possible causes of the validation–leaderboard gap 30
6.5 Improvement strategies 31
6.6 Strengths and weaknesses of the final solution 31
Task 7: Conclusion and Reproducibility 32
References 33

Summary

This report documents an end-to-end deep-learning pipeline for an eight-class image-classification task (cat or dog × eye, face, mouth or nose) with 63,660 labelled training images and 29,443 unlabelled test images. A shared tf.data pipeline feeds two ImageNet-pretrained backbones, EfficientNetB0 and ResNet50V2, each trained in two stages (frozen head, then partial fine-tuning) and combined by probability averaging. Validation accuracy was 99.43%, 99.51% and 99.64% (EfficientNetB0, ResNet50V2, two-model ensemble). A later experiment adding weight decay and label smoothing cut EfficientNetB0 errors and gave the best validation result (35 errors, 99.73%, four regularised models). Six Kaggle submissions scored 0.9767 to 0.9804 on the public leaderboard (the sixth is an exploratory ten-model ensemble), and the model with the best validation result was not the best on the leaderboard. The report also analyses the learning curves, the concentration of errors in cat nose/mouth confusion and in tiny or uninformative crops, a clean rerun showing about ten errors of run-to-run variation, and a validation-to-leaderboard gap of 1.7 to 1.9 points, and it states which conclusions the evidence supports and which remain hypotheses.

Task 1: Dataset Exploration and Understanding

1.1 Classification Problem and Dataset

The task is eight-class classification of cropped cat and dog facial-feature images from the ENGE707 Kaggle competition: two species (cat, dog) by four facial features (eye, face, mouth, nose), giving CatEye, DogEye, CatFace, DogFace, CatMouth, DogMouth, CatNose and DogNose. The dataset is a compressed archive with two image directories and two CSV files. The training directory holds 63,660 labelled JPEG images with five-digit zero-padded filenames (for example 00001.jpg) that correspond to the image_id field in train_labels.csv, which supplies the label for every training image (Figure 1). The test directory holds 29,443 unlabelled images, matching the 29,443 rows of sample_submission.csv, which defines the required submission columns (image_id, class). The file counts and the final filename (63660.jpg) align exactly with the two CSV files, and no files failed to extract.

```
[1]: import pandas as pd

     labels = pd.read_csv("../data/train_labels.csv")
     print(labels.shape)
     labels.head()
```

```
(63660, 2)
```

[1]:

	image_id	class
0	1	DogEye
1	2	DogMouth
2	3	DogFace
3	4	DogNose
4	5	DogNose

Figure 1: Loading train_labels.csv and previewing the first five rows.

1.2 Class Distribution and Balance

Class counts are imbalanced (Table 1, Figures 2 and 3). CatEye (13,884 images; 21.8%) and DogEye (11,580; 18.2%) have about double the images of the other classes of the same species, consistent with each photograph contributing two eye crops but only one face, mouth and nose crop. The largest class is about 2.4 times the smallest (DogNose), so accuracy alone could hide weak classes and per-class metrics are reported. In practice the Eye classes had the highest recall (Section 5.1), so the imbalance did not harm training.

Class	Training images	Proportion
CatEye	13,884	21.8%
DogEye	11,580	18.2%
CatFace	6,942	10.9%
CatMouth	6,942	10.9%
CatNose	6,942	10.9%
DogFace	5,790	9.1%
DogMouth	5,790	9.1%
DogNose	5,790	9.1%

Table 1: Number and proportion of training images per class.

```
import matplotlib.pyplot as plt

class_counts.sort_values(ascending=False).plot(kind='bar', figsize=(8,5))
plt.title("Class Distribution in Training Set")
plt.ylabel("Number of Images")
plt.xlabel("Class")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Figure 2: Bar chart of training image counts across the eight target classes.

```
[2]: class_counts = labels['class'].value_counts()
     print(class_counts)
```

```
class
CatEye      13884
DogEye      11580
CatNose      6942
CatFace      6942
CatMouth     6942
DogMouth     5790
DogFace      5790
DogNose      5790
Name: count, dtype: int64
```

Figure 3: Per-class image counts computed from train_labels.csv.

1.3 Representative Sample Images

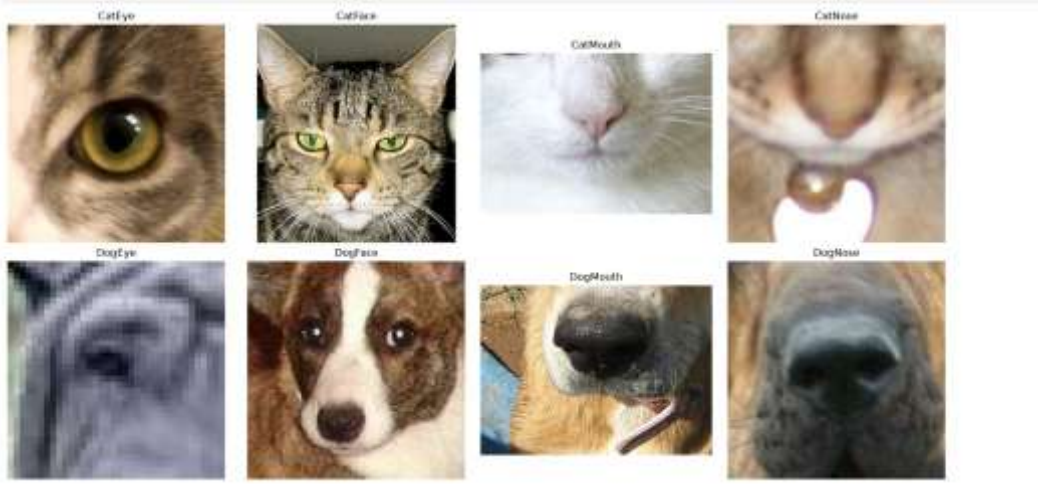


Figure 4: Representative sample image from each of the eight target classes.

One image per class (Figure 4) confirms that the labels match the content: Face images show a whole, recognisable face, and Eye, Mouth and Nose images show correctly cropped close-ups of that feature. The DogEye example is visibly low-resolution compared with the DogFace example. Because only eight images were inspected, this is a sanity check on what the labels mean, not evidence that every label is correct (Sections 1.4 and 6.1). The code that produced the figure is shown in Figure 5.

```
[4]: import os
     from PIL import Image

     img_dir = "../data/train/train"

     fig, axes = plt.subplots(2, 4, figsize=(16, 8))
     classes = sorted(labels['class'].unique())

     for ax, cls in zip(axes.flat, classes):
         sample_id = labels[labels['class'] == cls].iloc[0]['image_id']
         filename = f"{sample_id:05d}.jpg"
         img = Image.open(os.path.join(img_dir, filename))
         ax.imshow(img)
         ax.set_title(cls)
         ax.axis('off')

     plt.tight_layout()
     plt.show()
```

Figure 5: Code used to generate the representative sample image grid (Figure 4).

1.4 Data Quality Assessment

Image resolution varies widely. A random sample of 200 training images ranged from 16 to 816 pixels per side (mean about 120), and about 24% had a side under 50 pixels, far below the 224×224 input of pretrained networks (Figure 6). Size depends on class (Table 2): Face images average 200–240 pixels per side, Mouth 90–125, Eye 80–86 and Nose 70–80, because Face images keep the whole photograph while the others are progressively tighter crops. The smallest classes therefore need the most upscaling. Figure 7 shows the sampling code and output, and Figure 8 the per-class code.

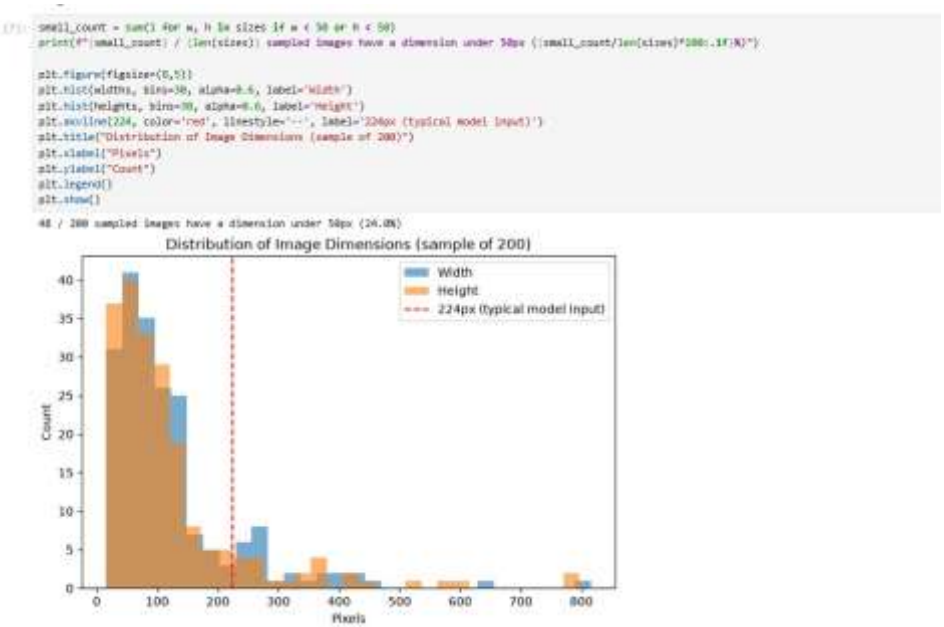


Figure 6: Distribution of training image widths and heights, relative to a 224-pixel reference input size.

```
[6]: import random

sample_files = random.sample(os.listdir(img_dir), 200)
sizes = []
for f in sample_files:
    with Image.open(os.path.join(img_dir, f)) as im:
        sizes.append(im.size)

widths, heights = zip(*sizes)
print("Width  - min:", min(widths), "max:", max(widths), "mean:", sum(widths)/len(widths))
print("Height - min:", min(heights), "max:", max(heights), "mean:", sum(heights)/len(heights))
```

```
Width  - min: 16 max: 816 mean: 120.64
Height - min: 16 max: 796 mean: 120.05
```

```
[ ]:
```

Figure 7: Minimum, maximum, and mean image width/height across a random sample of 200 training images.

Class	Mean width (px)	Mean height (px)
DogFace	206.5	206.0
CatFace	203.5	241.3
CatMouth	126.7	88.3
DogMouth	125.6	92.9
CatEye	86.1	86.1
DogEye	80.7	80.7
CatNose	80.6	80.1
DogNose	71.5	71.5

Table 2: Mean image width and height per class, based on a stratified sample of 100 images per class.

```
[8]: labels['width'] = None
labels['height'] = None

sample_per_class = labels.groupby('class').sample(n=100, random_state=42)

dims = []
for _, row in sample_per_class.iterrows():
    filename = f"{row['image_id']:05d}.jpg"
    with Image.open(os.path.join(img_dir, filename)) as im:
        dims.append({'class': row['class'], 'width': im.size[0], 'height': im.size[1]})

dims_df = pd.DataFrame(dims)
dims_df.groupby('class')[['width', 'height']].mean().sort_values('width', ascending=False)
```

[8]:

class	width	height
DogFace	206.51	206.01
CatFace	203.50	241.25
CatMouth	126.74	88.28
DogMouth	125.57	92.88
CatEye	86.12	86.12
DogEye	80.68	80.68
CatNose	80.58	80.07
DogNose	71.54	71.54

```
[ ]:
```

Figure 8: Code and output for per-class mean image dimensions (Table 2).

Every image in both directories was opened and checked with Pillow's Image.verify(). None of the 63,660 training or 29,443 test images was corrupted or unreadable, so none needed excluding or repairing (Figures 9 and 10).

```
[9]: corrupted = []
for f in os.listdir(img_dir):
    try:
        with Image.open(os.path.join(img_dir, f)) as im:
            im.verify()
    except Exception as e:
        corrupted.append((f, str(e)))

print(f"{len(corrupted)} corrupted/unreadable images out of {len(os.listdir(img_dir))}")
corrupted[:10]

0 corrupted/unreadable images out of 63660
[9]: []
```

Figure 9: Corrupted/unreadable file check across all 63,660 training images (result: 0).

```
[10]: test_dir = "../data/test/test"
corrupted_test = []
for f in os.listdir(test_dir):
    try:
        with Image.open(os.path.join(test_dir, f)) as im:
            im.verify()
    except Exception as e:
        corrupted_test.append((f, str(e)))

print(f"{len(corrupted_test)} corrupted/unreadable images out of {len(os.listdir(test_dir))}")
corrupted_test[:10]

0 corrupted/unreadable images out of 29443
[10]: []

[ ]:
```

Figure 10: Corrupted/unreadable file check across all 29,443 test images (result: 0).

Duplicates were searched on all 63,660 labelled images in two ways (Figures 11 and 12). Hashing the file bytes (MD5) found 30 byte-identical files in 15 groups (0.05% of labelled images, none with conflicting labels), and 3 validation images have an identical copy in the training subset. Near-duplicates were found with a 64-bit difference hash (images shrunk to 9×8 greyscale, adjacent pixels compared), counting pairs that differ in at most 3 bits and excluding flat images (fewer than 8 or more than 56 set bits) that hash alike regardless of content. This gave 521 pairs, 75 with different labels; these may reflect label ambiguity or hash coincidence on low-detail crops and were not inspected individually. In total 143 of the 12,732 validation images (1.12%) have an exact or near duplicate in the training subset, and none were removed; Section 6.4 measures their effect on validation accuracy. Overlap between labelled and test images was not checked.

```
[1]: import sys; sys.path.append("../src")
import hashlib, numpy as np, pandas as pd, matplotlib.pyplot as plt
import matplotlib.image as mpimg
from PIL import Image
import pipeline

df = pipeline.labels.copy()
df["split"] = np.where(df["filename"].isin(pipeline.val_df["filename"]), "val", "train")

md5s, hashes, blurs, sizes = [], [], [], []
for i, f in enumerate(df["filename"]):
    p = pipeline.TRAIN_DIR / f
    md5s.append(hashlib.md5(p.read_bytes()).hexdigest())
    im = Image.open(p).convert("L")
    sizes.append(im.size)
    g = np.asarray(im.resize((9, 8), Image.BILINEAR), dtype=np.int16)
    hashes.append(int(np.packbits((g[:, 1:] > g[:, :-1]).flatten()).view(">u8")[0]))
    a = np.asarray(im.resize((64, 64), Image.BILINEAR), dtype=np.float32)
    lap = 4 * a[1:-1, 1:-1] - a[:-2, 1:-1] - a[2:, 1:-1] - a[1:-1, :-2] - a[1:-1, 2:]
    blurs.append(float(lap.var()))
    if (i + 1) % 10000 == 0: print(i + 1, "images done")

df["md5"], df["blur"] = md5s, blurs
df["width"], df["height"] = zip(*sizes)
H = np.array(hashes, dtype=np.uint64)
df["dhash"] = [f"{h:016x}" for h in hashes]

# --- exact duplicates (identical files) ---
dup = df[df.duplicated("md5", keep=False)]
train_md5 = set(df.loc[df["split"] == "train", "md5"])
print("EXACT: files with an identical copy:", len(dup), "| groups:", dup["md5"].nunique(),
      "| groups with conflicting labels:", int((dup.groupby("md5")["class"].nunique() > 1).sum()))
print("EXACT: validation images with an identical copy in train:",
      int(df[(df["split"] == "val")]["md5"].isin(train_md5).sum()))

# --- near-duplicates (64-bit difference hash, at most 3 differing bits; flat images skipped) ---
pc = np.bitwise_count(H)
idx = np.where((pc >= 8) & (pc <= 56))[0]; Hs = H[idx]
pairs = []
for s in range(0, len(Hs), 256):
    d = np.bitwise_count(Hs[s:s + 256, None] ^ Hs[None, :])
    ii, jj = np.nonzero(d <= 3)
    keep = (ii + s) < jj
    pairs.append(np.c_[idx[ii[keep] + s], idx[jj[keep]]])
pairs = np.vstack(pairs)
a, b = pairs[:, 0], pairs[:, 1]
md, cl, sp = df["md5"].values, df["class"].values, df["split"].values
near = pairs[md[a] != md[b]]; na, nb = near[:, 0], near[:, 1]
print("\nNEAR: pairs:", len(near), "| pairs with different labels:", int((cl[na] != cl[nb]).sum()))
cross = sp[a] != sp[b]
val_hit = set(np.where(sp[a[cross]] == "val", a[cross], b[cross]))
print("NEAR+EXACT: validation images with a (near-)duplicate in train:", len(val_hit),
```

Figure 11: Duplicate, near-duplicate and blur check (code, part 1).



Figure 12: Duplicate, near-duplicate and blur check (code, part 2, and output).

Blur was scored as the variance of the Laplacian of a 64×64 greyscale copy (lower means blurrier); quantiles over the labelled set (5/25/50/75%) are 47.5, 206.8, 489.8 and 958.2. Blur depends on crop type: class medians are lowest for DogEye (262), CatNose (284) and DogNose (301) and highest for CatFace (994), matching the resolution pattern in Table 2. The 12 blurriest images (Figure 13) are near-featureless patches, consistent with the non-informative crops found in Section 6.1.

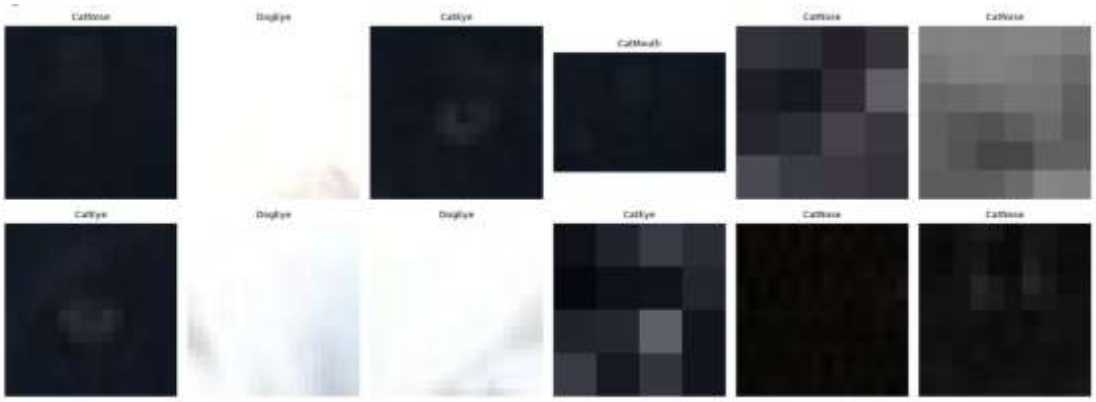


Figure 13: The 12 blurriest training images by Laplacian variance (titles show the labelled class).

Eight sample images matched their labels (Figure 4), but a sample that small cannot show that labels are clean: the error analysis (Section 6.1) later found noisy crops and ambiguous nose/mouth labels, including cat nose and mouth crops that can be genuinely ambiguous.

1.5 Dataset Usage Strategy

The 63,660 labelled images are split into stratified training and validation subsets (Section 2.1), so that performance can be monitored during training without exposing the model to the test set. The 29,443 test images are used only to generate the predictions submitted to Kaggle, and never in training, validation or hyperparameter selection, so the leaderboard gives an unbiased held-out measure (Section 5).

Task 2: Data Preprocessing and Pipeline Preparation

2.1 Label Encoding and Train/Validation Split

Class names were encoded alphabetically (CatEye = 0 to DogNose = 7) instead of by first appearance, so the mapping is deterministic across notebook restarts and between training and inference (Figure 14). The image_id values are plain integers while filenames are zero-padded (00001.jpg), so a filename column was built explicitly from image_id (Figure 15). The 63,660 labelled images were split 80/20 (50,928 training, 12,732 validation) with scikit-learn's train_test_split, stratified by class and seeded with random_state 42. Stratification was chosen because of the imbalance in Section 1.2: a non-stratified split could give a validation set with different class proportions by chance, making validation a less reliable estimate. Class proportions matched between the two subsets to within 0.01 percentage points (Figure 16).

```
[1]: import pandas as pd

labels = pd.read_csv("../data/train_labels.csv")

# Sort alphabetically for a consistent, reproducible mapping
class_names = sorted(labels['class'].unique())
class_to_idx = {name: idx for idx, name in enumerate(class_names)}
idx_to_class = {idx: name for name, idx in class_to_idx.items()}

labels['label'] = labels['class'].map(class_to_idx)

print(class_to_idx)
labels.head()
```

```
{'CatEye': 0, 'CatFace': 1, 'CatMouth': 2, 'CatNose': 3, 'DogEye': 4, 'DogFace': 5, 'DogMouth': 6, 'DogNose': 7}
```

[1]:

	image_id	class	label
0	1	DogEye	4
1	2	DogMouth	6
2	3	DogFace	5
3	4	DogNose	7
4	5	DogNose	7

Figure 14: Label encoding of the eight class names into a numerical class_to_idx mapping.

```
[2]: labels['filename'] = labels['image_id'].apply(lambda x: f"{x:05d}.jpg")
labels.head()
```

[2]:

	image_id	class	label	filename
0	1	DogEye	4	00001.jpg
1	2	DogMouth	6	00002.jpg
2	3	DogFace	5	00003.jpg
3	4	DogNose	7	00004.jpg
4	5	DogNose	7	00005.jpg

Figure 15: Construction of the zero-padded filename column from image_id.

```
[3]: from sklearn.model_selection import train_test_split

train_df, val_df = train_test_split(
    labels,
    test_size=0.2,
    stratify=labels['label'],
    random_state=42
)

print(f"Train: {len(train_df)} images")
print(f"Val:   {len(val_df)} images")

# Confirm class proportions match between the two sets
print("\nTrain class proportions:")
print(train_df['class'].value_counts(normalize=True).sort_index())
print("\nVal class proportions:")
print(val_df['class'].value_counts(normalize=True).sort_index())
```

```
Train: 50928 images
Val:   12732 images

Train class proportions:
class
CatEye      0.218092
CatFace     0.109056
CatMouth    0.109056
CatNose     0.109036
DogEye      0.181904
DogFace     0.090952
DogMouth    0.090952
DogNose     0.090952
Name: proportion, dtype: float64

Val class proportions:
class
CatEye      0.218112
CatFace     0.109017
CatMouth    0.109017
CatNose     0.109095
DogEye      0.181904
DogFace     0.090952
DogMouth    0.090952
DogNose     0.090952
Name: proportion, dtype: float64
```

Figure 16: Stratified 80/20 train/validation split and verification of matching class proportions.

2.2 Resizing, Normalisation, Data Loading and Consistent Test Pipeline

All images are resized to 224×224 pixels, the native input size of the ImageNet-pretrained backbones, and rescaled from [0, 255] to [0, 1], which is standard practice for stabilising gradient-based optimisation (Figure 17). A pretrained-compatible size was judged more important than limiting upscaling, since it allows pretrained weights to be used instead of training from scratch on a noisy, imbalanced dataset. The cost is that the smallest classes (CatNose and DogNose average only 70–86 pixels per side) are upscaled substantially without gaining real detail. Each model converts the shared [0, 1] range to its backbone's expected range internally (Section 3.2). Images are streamed with tf.data instead of being held in memory, which would mean keeping tens of thousands of 224×224×3 arrays at once: batch size 32, parallel decoding (num_parallel_calls=AUTOTUNE) and prefetching, so the CPU prepares the next batch while the GPU trains (Figure 18). Shuffling is applied to the training set only. The test pipeline uses identical resizing and scaling but no shuffling, so predictions stay aligned with the image_id order of sample_submission.csv, and no augmentation, so the test set measures unaltered performance (Figures 19 and 20).



Figure 17: Image loading, resizing to 224×224, and normalisation function, tested on a single image.

[5]:
```
BATCH_SIZE = 32

def make_dataset(df, shuffle=False):
    filenames = df['filename'].values
    labels_arr = df['label'].values
    ds = tf.data.Dataset.from_tensor_slices((filenames, labels_arr))
    if shuffle:
        ds = ds.shuffle(buffer_size=len(df), seed=42)
    ds = ds.map(load_and_preprocess, num_parallel_calls=tf.data.AUTOTUNE)
    ds = ds.batch(BATCH_SIZE)
    ds = ds.prefetch(tf.data.AUTOTUNE)
    return ds

train_ds = make_dataset(train_df, shuffle=True)
val_ds = make_dataset(val_df, shuffle=False)

# Quick check: pull one batch and confirm shapes
for images, batch_labels in train_ds.take(1):
    print("Batch images shape:", images.shape)
    print("Batch labels shape:", batch_labels.shape)
    print("Sample labels:", batch_labels.numpy()[:10])
```

```
Batch images shape: (32, 224, 224, 3)
Batch labels shape: (32,)
Sample labels: [7 6 0 2 3 5 6 6 1 7]
```

[ ]:

Figure 18: Batched, shuffled, and prefetched tf.data pipeline for the training and validation sets, with a verified batch shape of (32, 224, 224, 3).

[13]:
```
import os

TEST_DIR = "../data/test/test"
sample_sub = pd.read_csv("../data/sample_submission.csv")

# Build the same zero-padded filename format used for train
sample_sub['filename'] = sample_sub['image_id'].apply(lambda x: f"{x:05d}.jpg")

print(sample_sub.shape)
sample_sub.head()
```

```
(29443, 3)
```

[13]:

	image_id	class	filename
0	1	DogEye	00001.jpg
1	2	CatNose	00002.jpg
2	3	CatNose	00003.jpg
3	4	DogFace	00004.jpg
4	5	DogMouth	00005.jpg

Figure 19: Construction of test-set filenames from sample_submission.csv, matching the training-set convention.

```
[14]: def load_test_image(filename):
          path = TEST_DIR + "/" + filename
          image = tf.io.read_file(path)
          image = tf.image.decode_jpeg(image, channels=3)
          image = tf.image.resize(image, [IMG_SIZE, IMG_SIZE])
          image = image / 255.0
          return image

      test_filenames = sample_sub['filename'].values

      test_ds = tf.data.Dataset.from_tensor_slices(test_filenames)
      test_ds = test_ds.map(load_test_image, num_parallel_calls=tf.data.AUTOTUNE)
      test_ds = test_ds.batch(BATCH_SIZE)
      test_ds = test_ds.prefetch(tf.data.AUTOTUNE)

      # Quick check
      for batch in test_ds.take(1):
          print("Test batch shape:", batch.shape)

Test batch shape: (32, 224, 224, 3)
```

Figure 20: Test-set loading pipeline (no labels, no shuffling, no augmentation), verified against the expected batch shape of (32, 224, 224, 3).

2.3 Data Augmentation and Leakage Check

Augmentation was applied to the training set only: horizontal flips, rotation of about ±18°, and mild brightness and contrast changes (Figure 22). Stronger transformations such as large rotations or random crops were avoided because the Eye and Nose crops are already tight and could be pushed out of frame; vertical flips were excluded because faces do not appear upside-down. An early version produced black images because RandomBrightness assumes a [0, 255] range; passing value_range=(0.0, 1.0) and adding tf.clip_by_value after the augmentation layer fixed this, and the result was checked visually against the originals (Figure 21). To check the split, the set of training filenames was intersected with the set of validation filenames, which gave zero overlap (Figure 23). This is a file-level check: it cannot detect different crops of the same photograph on both sides of the split, which Section 6.4 discusses as a possible source of optimism in the validation score. The hash search in Section 1.4 found 143 validation images with a duplicate in training; none were removed, because re-scoring the predictions without them changed validation accuracy by 0.01 points or less (Section 6.4).

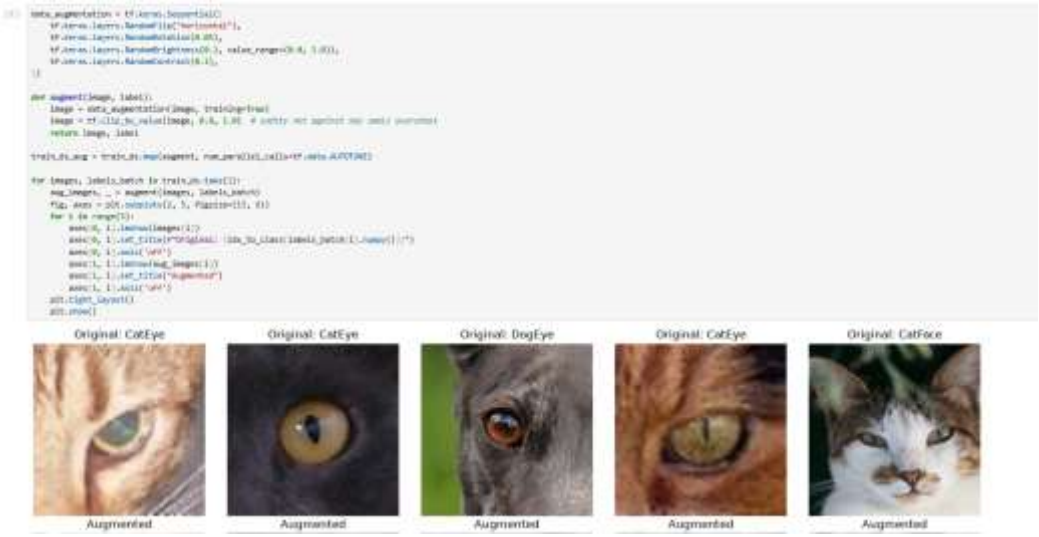


Figure 21: Augmentation pipeline code, corrected to operate in the [0, 1] pixel value range.



Figure 22: Visual comparison of five original training images (top) against their augmented counterparts (bottom), confirming the augmentation is subtle and preserves the classified feature.

```
[6]: overlap = set(train_df['filename']) & set(val_df['filename'])
     print(f"Overlapping filenames between train and val: {len(overlap)}")

     Overlapping filenames between train and val: 0

[ ]:
```

Figure 23: Verification that zero filenames overlap between the training and validation sets.

Task 3: Model Selection and Pipeline Development

3.1 Choice of architecture and justification

The task is fine-grained in two ways: the model must separate species and facial region, and many crops are tiny (Section 1.4). Three model families were considered: a custom convolutional neural network (CNN) trained from scratch, a Vision Transformer (ViT), and an ImageNet-pretrained CNN adapted by transfer learning. Transfer learning was selected. A from-scratch CNN would have to learn edges, textures and shapes from 50,928 noisy, often low-resolution images, whereas ImageNet features (Deng et al., 2009) transfer to related tasks (Yosinski et al., 2014; Kornblith et al., 2019), and ImageNet's many cat and dog breeds make the later features relevant. A ViT (Dosovitskiy et al., 2021) is costlier to fine-tune and its patch tokenisation offers no clear advantage for 16-pixel crops upscaled to 224. Neither a custom CNN nor a ViT was trained, so this choice rests on the literature and the compute budget, not a head-to-head test.

EfficientNetB0 (Tan & Le, 2019) was the primary model: about 4.05 million backbone parameters and 35–41 seconds per epoch on the RTX 3080 used. **ResNet50V2** (He et al., 2016) was the comparison backbone. It differs in almost every respect (23.56 million parameters, pre-activation residual blocks, a different expected input range), so its errors were expected to differ, making it a good ensemble partner (Dietterich, 2000). The ensembles average the models' predicted probabilities.

3.2 Architecture

Both networks share one head design (Figure 24, Table 3): a rescaling layer, the pretrained backbone without its classification layer, global average pooling, dropout of 0.3 and a dense softmax layer with eight units. The rescaling layer is the one deliberate difference. The shared pipeline delivers [0, 1] pixels, but Keras EfficientNet normalises internally and expects 0–255, whereas Keras ResNet50V2 expects [−1, 1]. Converting inside each model keeps the data pipeline identical, so performance differences reflect the networks, not the preprocessing. Global average pooling keeps the head small (10,248 parameters for EfficientNetB0), reducing the capacity to memorise. Dropout (Srivastava et al., 2014) zeroes 30% of the pooled features during training only; 0.3 is a conventional value and was **not tuned**. Softmax suits eight mutually exclusive classes.

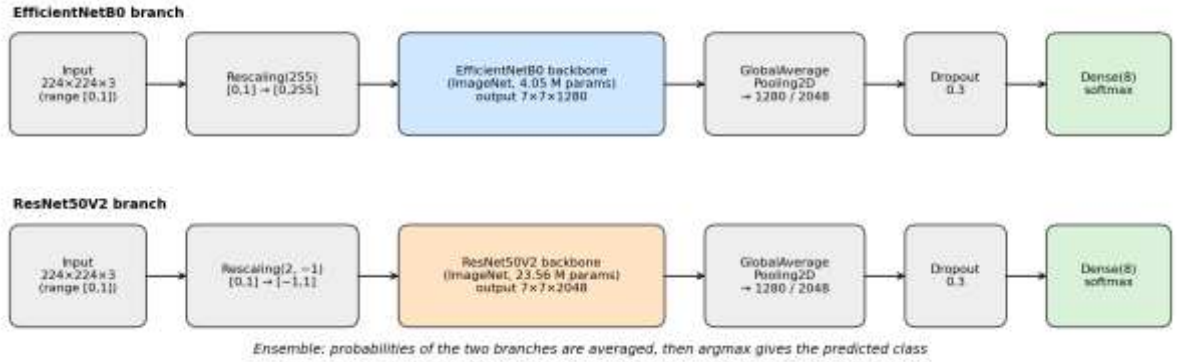


Figure 24: Architecture of the two branches and the probability-averaging ensemble.

Component	EfficientNetB0 model	ResNet50V2 model
Rescaling layer	Rescaling(255): [0,1] → [0,255]	Rescaling(scale 2, offset −1): [0,1] → [−1,1]
Backbone output	7 × 7 × 1280 (4,049,571 params)	7 × 7 × 2048 (23,564,800 params)
Pooling / dropout	GlobalAveragePooling2D → 1,280 values; Dropout 0.3	GlobalAveragePooling2D → 2,048 values; Dropout 0.3
Output layer	Dense(8, softmax): 10,248 params	Dense(8, softmax): 16,392 params

Component	EfficientNetB0 model	ResNet50V2 model
Total parameters	4,059,819	23,581,192
Trainable, stage 1	10,248 (head only)	16,392 (head only)
Trainable, stage 2	3,140,132 (blocks 6, 7 and top convolution)	14,966,792 (conv5 stage)

Table 3: Layer summary and parameter counts (from model.summary()).

3.3 Loss, Optimiser and Hyperparameters

Table 4 gives the training configuration and the reason for each choice. Sparse categorical cross-entropy suits integer labels (Section 2.1), and Adam (Kingma & Ba, 2015) adapts the step size per parameter. Class weighting was not applied because the Eye classes already had the highest recall (Section 5.1). The regularised variant in Section 4.5 swaps Adam for AdamW with weight decay 0.05 and adds label smoothing of 0.1.

Setting	Value	Rationale
Input size	224 × 224 × 3	Native size of both ImageNet backbones; small crops are upscaled (Section 2.2)
Output layer	Dense(8), softmax	One probability per mutually exclusive class
Loss	Sparse categorical cross-entropy (regularised variant: categorical cross-entropy with label smoothing 0.1)	Integer labels with softmax outputs
Optimiser	Adam (regularised variant: AdamW, weight decay 0.05)	Adaptive per-parameter steps; robust default
Learning rate	Stage 1: 1e-3; stage 2: 1e-4	A freshly initialised head tolerates a large step; pretrained layers should change gently, so the rate is cut tenfold
Batch size	32	1,592 optimisation steps per epoch; fits GPU memory for both backbones
Epochs	Stage 1: 5; stage 2: up to 12	Stage-1 gains had flattened by epoch 5; stage 2 is bounded and controlled by callbacks
Regularisation	Dropout 0.3, augmentation, frozen BatchNorm, early stopping, learning-rate reduction; weight decay and label smoothing in the regularised variant	Discussed in Section 4
Callbacks (stage 2)	ModelCheckpoint (val accuracy); EarlyStopping (val loss, patience 3, restore best weights); ReduceLROnPlateau (factor 0.3, patience 1, minimum 1e-6)	Keep the best weights, stop when validation loss stalls, and shrink the step size when progress stops

Table 4: Model and training configuration.

3.4 Implementation and Reproducibility

All data handling lives in one module, src/pipeline.py, imported by every notebook so that the training, validation and test paths cannot drift apart. It implements the alphabetical label encoding, the stratified 80/20 split, the tf.data loaders, the training-only augmentation (horizontal flip, rotation factor 0.05 or roughly ±18°, brightness 0.1, contrast 0.1) and the identical resize-and-scale step for all three splits. The environment was Python 3.12, TensorFlow 2.21.0 (Abadi et al., 2016), Keras 3.15.1 (Chollet et al., 2015), NumPy 2.5.3 (Harris et al., 2020), pandas 3.0.6 (McKinney, 2010), scikit-learn 1.9.1 (Pedregosa et al., 2011), Matplotlib 3.11.2 (Hunter, 2007) and Pillow 12.3.0 on an NVIDIA RTX 3080, recorded in requirements.txt. Each submission was checked against sample_submission.csv before upload (29,443 rows, columns image_id and class, same ids in the same order); the id column is taken from the original CSV, not from the zero-padded filenames. The split seed is fixed and tf.keras.utils.set_random_seed(42)

seeds Python, NumPy and TensorFlow, but GPU training is not bit-for-bit deterministic. A clean rerun (Section 6.3) confirmed this, so exact reproduction of every number should not be expected. The run-1 files that produced the first Kaggle scores were archived so the submitted CSVs match the reported results.

Task 4: Training Strategy and Overfitting/Underfitting Management

4.1 Two-stage transfer learning

Training followed a freeze-then-fine-tune schedule. In **stage 1** the pretrained backbone was frozen (called with training=False so its BatchNorm layers stayed in inference mode) and only the new head was trained for five epochs at a learning rate of 1e-3. In **stage 2** the top of each backbone was unfrozen and the model recompiled at 1e-4: blocks 6 and 7 and the top convolution for EfficientNetB0 (3,140,132 of 4,059,819 parameters trainable) and the conv5 stage for ResNet50V2 (14,966,792 of 23,581,192). Only the top is unfrozen because early layers capture generic edges and textures, while later layers are task-specific and benefit most from adaptation (Yosinski et al., 2014). BatchNorm layers stay frozen because their ImageNet statistics are stable and, with a small learning rate, leaving them fixed removes one source of instability (Ioffe & Szegedy, 2015).

4.2 Stage 1: frozen backbone (mild underfitting)

Validation accuracy sits above training accuracy throughout stage 1 (Table 5). This is not leakage: training figures are averaged over each epoch with augmentation and dropout active, whereas validation uses clean images with dropout off. The informative signal is the plateau. Accuracy rose only 0.0003 between epochs 3 and 4 (0.9434 to 0.9437) and ended at 0.9457 while loss kept falling slowly. A frozen backbone can only re-weight ImageNet features that were never trained to separate a cat nose from a cat mouth crop. This is mild **underfitting** (limited capacity to adapt, not too little training) and it motivated stage 2. ResNet50V2 showed the same pattern at a higher level (validation accuracy 0.9561 to 0.9687, validation loss 0.1242 to 0.0966).

Epoch	Train acc.	Val. acc.	Val. loss
1	0.8552	0.9265	0.2161
3	0.9095	0.9434	0.1635
5	0.9122	0.9457	0.1517

Table 5: EfficientNetB0 stage 1 (frozen backbone), key epochs.

4.3 Stage 2: fine-tuning and the onset of overfitting

Unfreezing the top blocks was the decisive change: validation accuracy jumped from 0.9457 to 0.9829 in the first stage-2 epoch and reached 0.9943 by epoch 12, a gain of 4.9 percentage points (Figure 25, Table 6). Overfitting also emerged. By epoch 12 training loss was 0.0055 against a validation loss of 0.0206, about four times larger, and from epoch 6 validation loss was nearly flat (0.021–0.022) while training loss kept falling. Validation accuracy did not degrade, so this is mild overfitting.

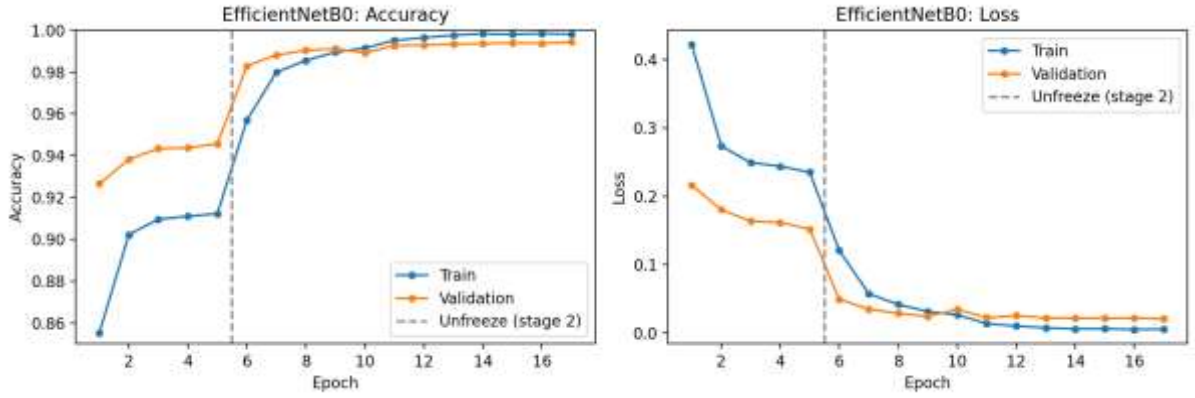


Figure 25: EfficientNetB0 training and validation accuracy and loss across both stages (dashed line: backbone unfrozen).

Epoch	Train acc.	Val. acc.	Train loss	Val. loss	Learning-rate event
1	0.9570	0.9829	0.1205	0.0492	1e-4 (start)
5	0.9914	0.9892	0.0262	0.0342	Cut to 3e-5 after val. loss rose
7	0.9964	0.9929	0.0099	0.0252	Cut to 9e-6
11	0.9984	0.9937	0.0049	0.0218	Cut to 2.7e-6
12	0.9981	0.9943	0.0055	0.0206	Best epoch, weights kept

Table 6: EfficientNetB0 stage 2, key epochs (epoch numbering restarts at 1).

Two mechanisms limited it. **ReduceLROnPlateau** cut the learning rate three times (to 3e-5, 9e-6 and 2.7e-6), each cut followed by equal or lower validation loss, which is direct evidence that the schedule helped; the rise in validation loss at epoch 5 (0.0342 from 0.0240) is the instability a smaller step corrects. **EarlyStopping** (validation loss, patience 3, best weights restored) never fired because validation loss kept reaching new lows, so it acted only as a safeguard. Stopping at the 12-epoch limit means the curve may not have fully converged (Section 6.5).

4.4 ResNet50V2: more capacity, more overfitting

ResNet50V2 followed the same recipe with 14.97 million trainable parameters in stage 2, almost five times as many as EfficientNetB0 (Figure 26). Validation accuracy rose from 0.9818 to 0.9940 by epoch 4, dipped to 0.9901 when the rate was cut at epoch 5, and peaked at 0.9951 at epoch 8, where validation loss was lowest (0.0189). The rate was reduced four times, to 1e-6, and early stopping fired after epoch 11, restoring the epoch-8 weights. The generalisation gap was wider than for EfficientNetB0: by epoch 11 training loss was 0.0021 against a validation loss of 0.0197 (nine times larger), and at epoch 8, the weights evaluated in Section 5, still about six times. The checkpoint on disk is the epoch-11 file chosen by validation accuracy, which differs by about four images; all reported ResNet results use the restored epoch-8 model.

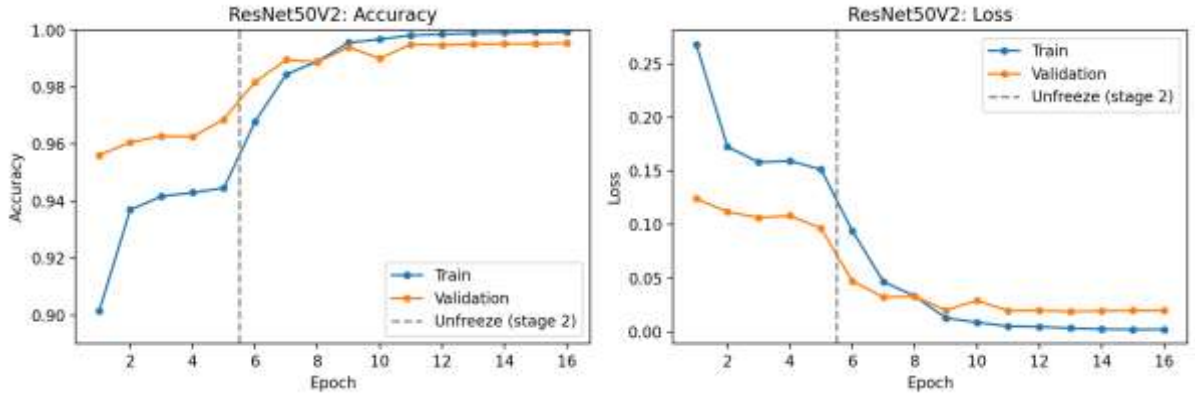


Figure 26: ResNet50V2 training and validation accuracy and loss across both stages.

4.5 Regularisation experiment: weight decay and label smoothing

Stage 2 showed the clearest overfitting (Sections 4.3 and 4.4), so a second experiment tested two standard remedies together. Stage 2 was rerun from the same stage-1 checkpoints with Adam replaced by AdamW (Loshchilov & Hutter, 2019) with weight decay 0.05, which penalises large weights, and with label smoothing of 0.1 (Szegedy et al., 2016), which softens each one-hot target so the model is not pushed towards extreme confidence on noisy or ambiguous crops (Section 6.1). The learning rate (1e-4), unfrozen layers, callbacks and 12-epoch limit were unchanged. Each backbone was trained with two seeds (42 and 7), giving four regularised models (Figures 27 to 30). Because the two changes were made together, their separate contributions are not measured.



Figure 27: Regularised EfficientNetB0, seed 42: stage-2 code and training log (AdamW weight decay 0.05, label smoothing 0.1); best epoch 12, validation accuracy 0.9953.

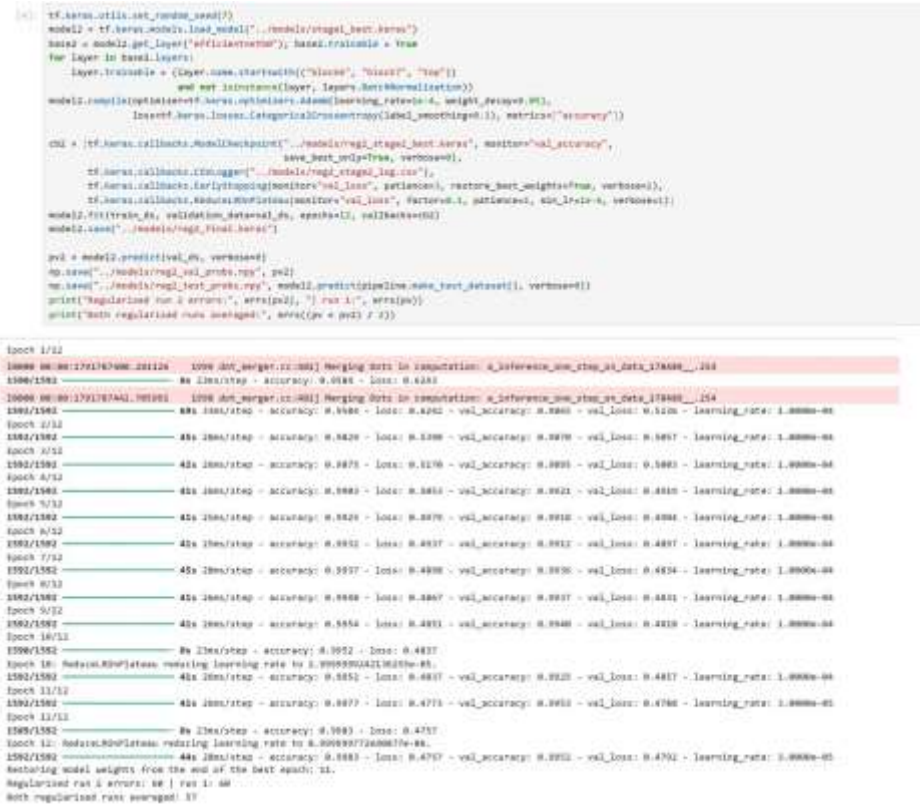


Figure 28: Regularised EfficientNetB0, seed 7: configuration, training log and validation errors (60, with 57 for the two-model average).

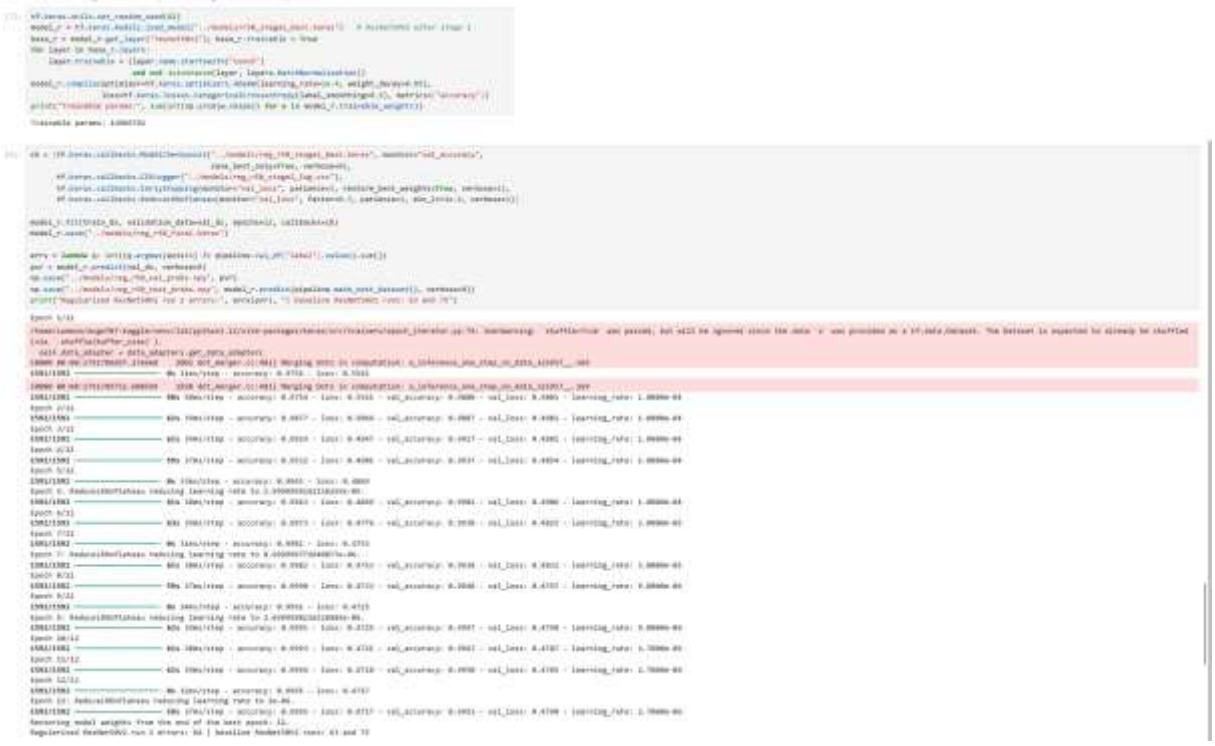


Figure 29: Regularised ResNet50V2, seed 42: configuration (14,966,792 trainable parameters), training log and validation errors (64).

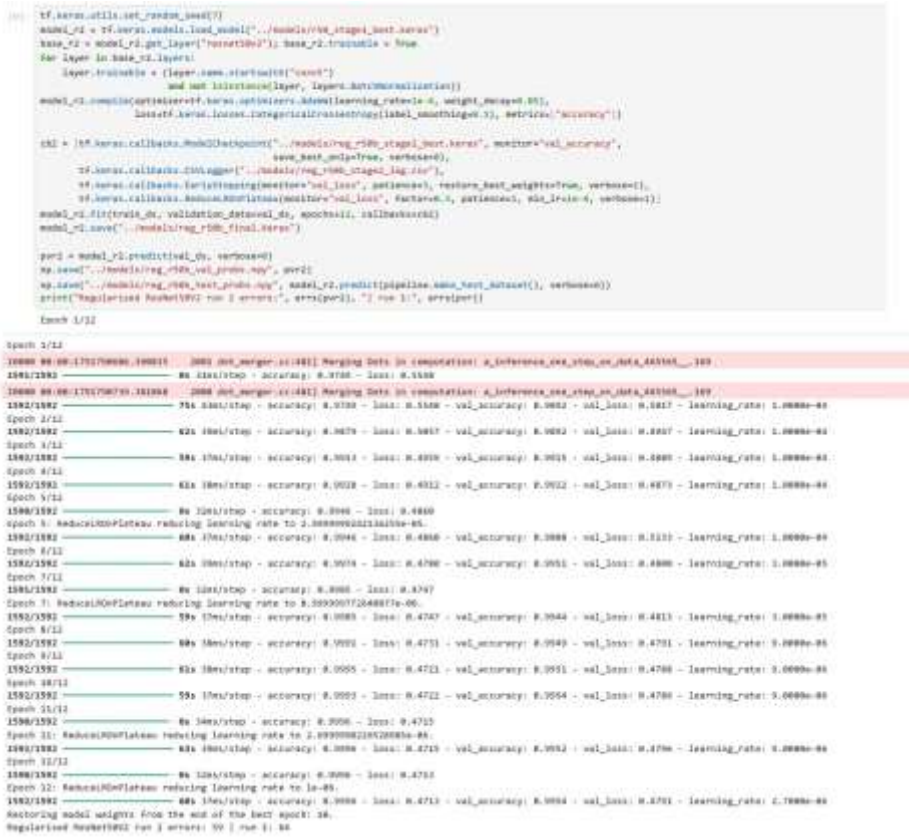


Figure 30: Regularised ResNet50V2, seed 7: configuration, training log and validation errors (59).

Two points matter when reading the logs. First, label smoothing puts a floor under the loss: with eight classes the cross-entropy cannot fall below about 0.47, so the regularised logs show losses near 0.47 instead of 0.01 to 0.02 and cannot be compared with the baseline loss values; results are therefore compared by validation errors. Second, the training-to-validation accuracy gap for EfficientNetB0 narrowed slightly at the best epoch, from 0.38 points (0.9981 against 0.9943) to 0.30 points (0.9983 against 0.9953).

Table 7 compares the baseline and regularised models. EfficientNetB0 improved in both seeds (73 and 75 errors fell to 60 and 60), but ResNet50V2 was mixed (63 became 64 in the first run and 75 became 59 in the second), so, given the run-to-run variation of about ten errors (Section 6.3), the single-model evidence for ResNet50V2 is weak. Averaging the four regularised models gave 35 validation errors (99.73%) against 43 for the four baseline models. A rule set before the ensemble was evaluated, to submit it only if it had at most 38 errors, was met (Figure 31).

Model	Baseline errors	Regularised errors
EfficientNetB0, first run (regularised: seed 42)	73	60
EfficientNetB0, second run (baseline: rerun; regularised: seed 7)	75	60
EfficientNetB0, two runs averaged	66	57
ResNet50V2, first run (regularised: seed 42)	63	64
ResNet50V2, second run (baseline: rerun; regularised: seed 7)	75	59
ResNet50V2, two runs averaged	59	55
Four-model ensemble (both backbones, two runs each)	43	35

Table 7: Validation errors (12,732 images) for baseline and regularised models.

```
[10]: y = pipeline.val_df["label"].values
errs = lambda q: int((q.argmax(axis=1) != y).sum())

base_r50_pair = (lv("backup_run1", "val_probs_r50.npy") + lv("backup_run2", "val_probs_r50.npy")) / 2
print("Regularised ResNet50V2 pair:", errs((pvr + pvr2) / 2), "| baseline ResNet50V2 pair:", errs(base_r50_pair))

e4r = (pv + pv2 + pvr + pvr2) / 4          # 2 regularised EfficientNet00 + 2 regularised ResNet50V2
n = errs(e4r)
print(f"Fully regularised 4-model ensemble: {n} errors | accuracy {1 - n / len(y):.4f} | baseline 4-model ensemble: 43")
print("Rule (submit only if <= 38 errors):", "MET" if n <= 38 else "NOT MET")

if n <= 38:
    t4r = sum(np.load(f"../models/{f}") for f in ["reg_test_probs.npy", "reg2_test_probs.npy",
                                                  "reg_r50_test_probs.npy", "reg_r50b_test_probs.npy"]) / 4
    sub = pd.DataFrame({"image_id": pipeline.test_df["image_id"].values,
                        "class": [pipeline.idx_to_class[i] for i in t4r.argmax(axis=1)]})
    sub.to_csv("../submissions/submission_v4_reg_ensemble4.csv", index=False)
    sample = pd.read_csv("../data/sample_submission.csv")
    print("Saved. Shape:", sub.shape, "| columns match:", list(sub.columns) == list(sample.columns),
          "| IDs match:", (sub["image_id"].values == sample["image_id"].values).all())

Regularised ResNet50V2 pair: 55 | baseline ResNet50V2 pair: 59
Fully regularised 4-model ensemble: 35 errors | accuracy 0.9973 | baseline 4-model ensemble: 43
Rule (submit only if <= 38 errors): MET
Saved. Shape: (29443, 2) | columns match: True | IDs match: True
```

Figure 31: Evaluation of the four regularised models: 35 errors (accuracy 0.9973), the submission rule (at most 38 errors) met, and submission_v4_reg_ensemble4.csv written and format-checked.

4.6 Summary of strategy changes and their justification

Partial unfreezing gave the largest gain (+4.9 points for EfficientNetB0, +2.6 for ResNet50V2), a tenfold lower rate with frozen BatchNorm protected pretrained features, and learning-rate reduction with early stopping controlled divergence. Weight decay with label smoothing then reduced validation errors further (35 against 43 for the four-model ensemble), although this did not carry over to the leaderboard (Section 5.3). An ablation of augmentation and dropout was not run, so their separate contributions are not measured (Section 6.3).

Task 5: Model Evaluation and Kaggle Submission

5.1 Validation Results

Models were evaluated on the 12,732-image validation set, which was never used for gradient updates. Because the classes are imbalanced, per-class precision, recall and F1 and a confusion matrix are reported alongside accuracy and loss (Tables 8 to 10, Figure 32). Validation data also drove checkpoint selection, early stopping and learning-rate reduction, so validation figures are slightly optimistic.

Model	Accuracy	Val. loss	Errors	Macro F1
EfficientNetB0	0.9943	0.0206	73	0.9940
ResNet50V2 (epoch 8)	0.9951	0.0189	63	0.9950
Ensemble (probability average)	0.9964	0.0138 (log loss)	46	–

Table 8: Validation performance (12,732 images, run 1). The ensemble figure is log loss because it averages probabilities.

Class	Support	EfficientNetB0			ResNet50V2		
		Precision	Recall	F1	Precision	Recall	F1
CatEye	2,777	0.9964	0.9960	0.9962	0.9957	0.9978	0.9968
CatFace	1,388	1.0000	1.0000	1.0000	1.0000	0.9986	0.9993
CatMouth	1,388	0.9842	0.9856	0.9849	0.9906	0.9892	0.9899
CatNose	1,389	0.9834	0.9806	0.9820	0.9849	0.9870	0.9860
DogEye	2,316	0.9953	0.9978	0.9966	0.9961	0.9948	0.9955
DogFace	1,158	0.9991	1.0000	0.9996	1.0000	0.9991	0.9996
DogMouth	1,158	0.9991	0.9965	0.9978	0.9966	0.9983	0.9974

DogNose	1,158	0.9957	0.9948	0.9952	0.9965	0.9940	0.9952
Macro avg	**12,732**	**0.9941**	**0.9939**	**0.9940**	**0.9951**	**0.9949**	**0.9950**

Table 9: Per-class precision, recall and F1 on the validation set (run 1), with class support.

Class	Support	Precision	Recall	F1
CatEye	2,777	0.9982	0.9986	0.9984
CatFace	1,388	1.0000	0.9993	0.9996
CatMouth	1,388	0.9878	0.9906	0.9892
CatNose	1,389	0.9885	0.9870	0.9878
DogEye	2,316	0.9983	0.9974	0.9978
DogFace	1,158	1.0000	1.0000	1.0000
DogMouth	1,158	0.9974	0.9991	0.9983
DogNose	1,158	0.9991	0.9974	0.9983
Macro avg	**12,732**	**0.9962**	**0.9962**	**0.9962**

Table 10: Per-class precision, recall and F1 of the ensemble (the submitted model) on the validation set (run 1).

Both single models classify more than 99.4% of validation images correctly, and the ensemble reaches 99.64% with 46 errors, against 73 for EfficientNetB0 and 63 for ResNet50V2. The two models shared 26 errors and 110 images were wrong for at least one, so about three-quarters of single-model mistakes were not shared, which is the diversity probability averaging exploits. Per-class results are uniformly strong. The Eye classes, with twice as many images, score F1 of 0.996–0.998, so the imbalance did not bias the models and class weighting was unnecessary. The weakest classes are CatNose (0.982, 0.986) and CatMouth (0.985, 0.990); the Face classes are near perfect. The ensemble, which is the submitted model, has the best macro F1 (0.9962 against 0.9940 and 0.9950) and the best F1 on six of eight classes; for CatFace and CatMouth it is within 0.001 of the better single model (Table 10). The confusion matrix shows the errors concentrated in one place: 18 CatMouth images predicted as CatNose and 19 CatNose as CatMouth for EfficientNetB0. The ensemble reduces these to 13 and 15, which are still 28 of its 46 errors (61%) (Figure 32, Section 6.1). Figure 33 shows the ensemble evaluation code and output.

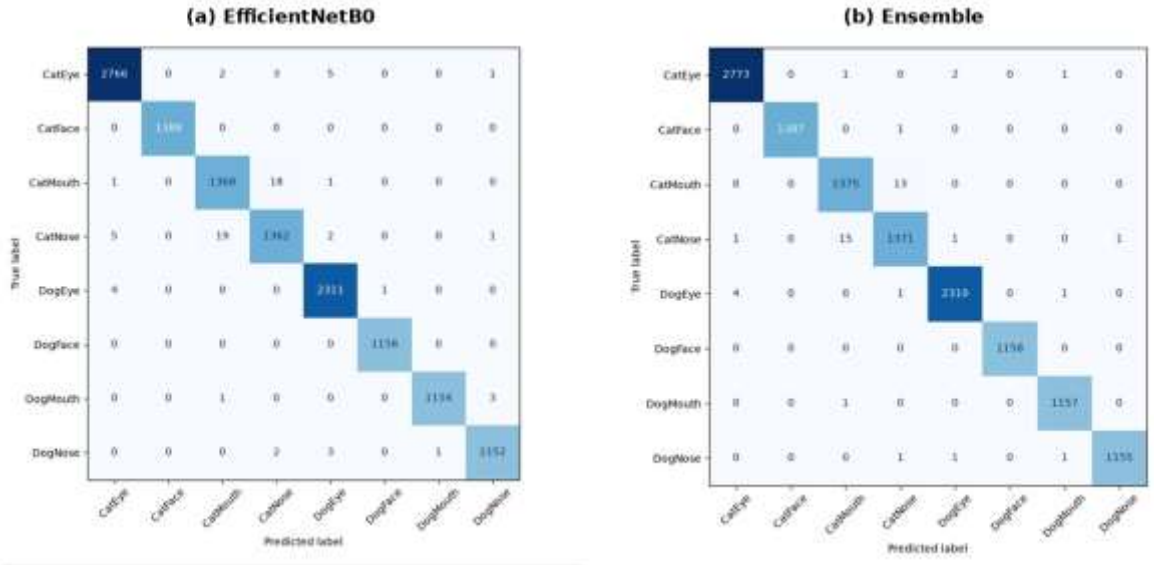


Figure 32: Validation confusion matrices (rows: true class; columns: predicted class) for (a) EfficientNetB0 and (b) the ensemble.



Figure 33: Ensemble evaluation code and output (46 errors, accuracy 0.9964).

5.2 Kaggle Submission and Leaderboard Evidence

Six submissions were made under the Kaggle name **Suemon Kwok**: v1 and v2 on 7 October 2026 and v3, v4a and v4b on 12 October 2026 and v5 on 21 October 2026 (Figures 34 to 36, Table 11). Version 1 (EfficientNetB0) scored **0.9767** and version 2 (the two-model ensemble) **0.9786**. Version 3 averages four baseline models (two runs each of EfficientNetB0 and ResNet50V2) to reduce training variance and scored **0.9790**. Version 4a (submission_v4_reg_ensemble4.csv) averages the four regularised models from Section 4.5 and scored **0.9784**. Version 4b (submission_v4_ensemble6.csv) averages six models (the four baseline models plus the two regularised EfficientNetB0 models) and scored **0.9800**. Version 4a met its pre-set submission rule (35 errors, at most 38). Version 4b did not meet its rule (submit only if validation errors fall below 43): it had 45, so it is an exploratory submission, not one chosen by validation. The two v4 files differ on 239 of the 29,443 test predictions (0.8%). Version 5 (submission_v5_exploratory_all10.csv) averages all ten models available: the eight models behind versions 3 and 4a (four baseline, four regularised) plus EfficientNetV2-S and ConvNeXt-Tiny (Section 6.3). It scored **0.9804**, the best public score, but its 37 validation errors were above its pre-set limit of 30, so it too is an exploratory submission, made to see how far the leaderboard score could be pushed. Its 0.0004 lead over version 4b is about 12 test images and inside run-to-run noise (Section 6.3), so it was not selected. Kaggle counts two submissions towards the final score, and v4b and v4a are the two selected (Figure 36). Only the competition data were used, with no external data and no manual labelling of test images.



Figure 34: Kaggle public leaderboard after submission v1 (EfficientNetB0), 4:33 pm, 7 October 2026. Other entrants' rows removed.



Figure 35: Kaggle public leaderboard after submission v2 (ensemble), 5:40 pm, 7 October 2026. Other entrants' rows removed.



Figure 36: Kaggle Submissions page showing all six submissions with their public scores and the two selected for the final score (ticked), taken on 21 October 2026.

Version	Models averaged	Val. errors	Val. accuracy	Kaggle public	Gap (points)
v1	EfficientNetB0	73	0.9943	0.9767	1.76
v2	EfficientNetB0 + ResNet50V2	46	0.9964	0.9786	1.78
v3	4 baseline models (2 runs each)	43	0.9966	0.9790	1.76
v4a	4 regularised models	35	0.9973	0.9784	1.89
v4b	6 models (4 baseline + 2 regularised EfficientNetB0)	45	0.9965	0.9800	1.65
v5	10 models (4 baseline + 4 regularised + EfficientNetV2-S + ConvNeXt-Tiny)	37	0.9971	0.9804	1.67

Table 11: Validation versus Kaggle public score for all six submissions (v4b and v5 missed their pre-set submission rules).

5.3 Comparing Validation and Leaderboard Performance

All six submissions scored between 1.65 and 1.89 points below validation. Sampling noise cannot explain this: the binomial standard error is about 0.07 points for 99.4% on 12,732 images and about 0.08 for 97.9% on 29,443, roughly 0.11 for the difference, so a gap of about 1.8 points is more than ten standard errors and points to a systematic difference between validation and test data. The model's confidence agrees. For EfficientNetB0, predictions with maximum probability below 0.9 were 0.83% of validation but 2.29% of test, below 0.99 they were 2.07% and 4.52%, and mean confidence fell from 0.9968 to 0.9924. Section 6.4 weighs two candidate explanations, neither tested: crops of the same photograph on both sides of the split, and a test set with more small or blurry crops. Exact and near duplicates do not explain it: removing the 143 flagged validation images changes accuracy by 0.01 points or less (Section 6.4).

The regularisation experiment adds a caution. Version 4a had the best validation result (35 errors against 43 for version 3) yet scored 0.0006 lower on the leaderboard (0.9784 against 0.9790), and the six-model version 4b, with more validation errors than either (45), scored 0.9800, above either. Validation errors and public score therefore did not rank the ensembles the same way. The exploratory ten-model version 5, with 37 validation errors, scored highest of all (0.9804), but only 0.0004 above version 4b. The five ensembles span only 0.0020 on the public score, which is similar to the +0.0019 v1-to-v2 gain that Section 6.3 shows is about the size of simply retraining, so the public differences between them are not evidence that one is

better, and the validation gain from regularisation is not shown to carry over to the test set. Validation accuracy should be read as an upper bound, and the leaderboard score as the better estimate of generalisation.

Task 6: Results Analysis and Improvement Strategies

6.1 Where the errors are

Of EfficientNetB0's 73 validation errors, 37 (51%) are CatMouth/CatNose swaps (18 + 19), whereas the dog equivalent accounts for only four. A plausible reason is that a cat's nose and mouth lie close together on a small muzzle, so a tight nose crop often contains part of the mouth; several crops in Figure 37 arguably contain both features. This is consistent with the data but was not tested directly.

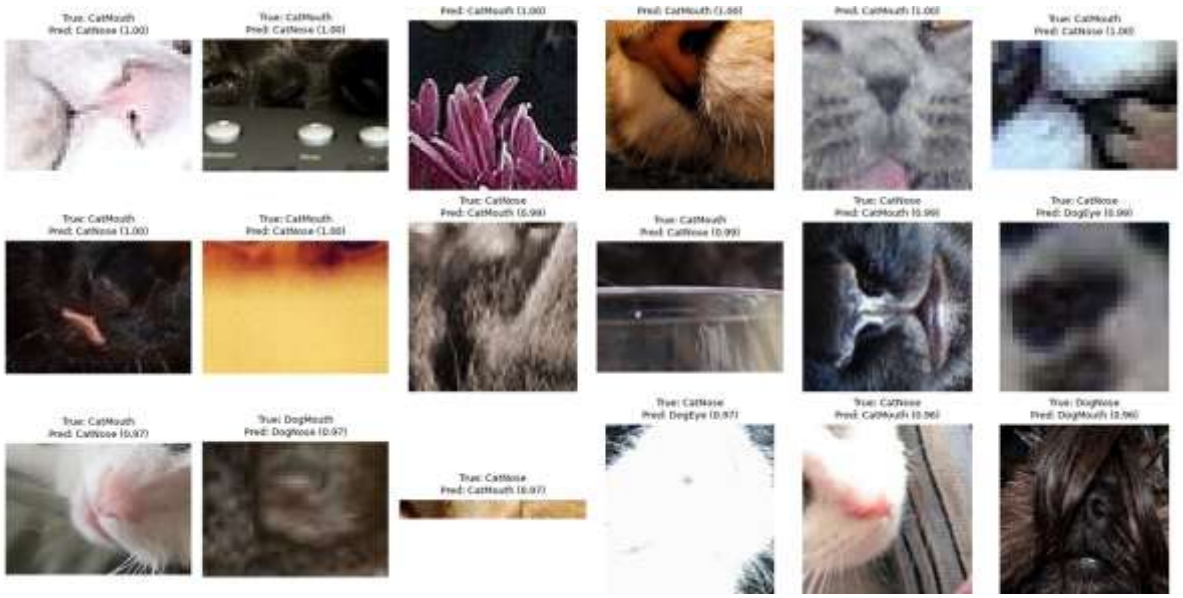


Figure 37: The most confident validation mistakes of EfficientNetB0 (true and predicted labels, with predicted probability). In the original plot the top title line is clipped for some panels in the first row.

The 24 most confident mistakes show three patterns. Some crops carry no usable information (a flower, part of a keyboard, a blank patch, a strip one pixel tall): extraction failures in the dataset, not model failures. Several nose and mouth crops show both features. Some are so pixelated that even a human would struggle. The model's confident answer is often as defensible as the label. This refines Section 1.4: the dataset contains noisy crops and ambiguous labels, so labels should not be assumed clean.

6.2 Error rate and image resolution

Error rate falls steeply as resolution rises (Table 12). Images with a shorter side of 64 pixels or less are 39% of the validation set but 68% of its errors, and the ≤32-pixel bucket has an error rate about seven times that of the 65–128 bucket. The rerun reproduces the pattern, so it is not an artefact of one run. Two caveats prevent reading this as proof that low resolution causes errors. Resolution is confounded with crop quality, because the junk crops are mostly tiny, and the rate flattens at 0.30–0.34% in the middle buckets, suggesting a floor from label noise that more detail would not remove. Upscaling adds no information (Section 2.2), so this floor is probably near what this approach can achieve without cleaner labels. Blur follows crop type rather than predicting difficulty: DogEye is as blurry as CatNose (Section 1.4) yet scores F1 above 0.995 for every model.

Shorter side (px)	Images	Errors	Error rate	Rerun error rate
≤ 32	1,001	23	2.30%	1.90%
33–64	3,925	27	0.69%	0.74%

Shorter side (px)	Images	Errors	Error rate	Rerun error rate
65–128	4,766	16	0.34%	0.36%
129–256	2,324	7	0.30%	0.39%
> 256	716	0	0.00%	0.14%

Table 12: EfficientNetB0 validation error rate by shorter image side (run 1; rerun in the last column).

6.3 Comparison of experiments and effectiveness of design choices

Table 13 supports six conclusions. **Fine-tuning mattered most**, lifting EfficientNetB0 from 0.9457 to 0.9943. **The backbones are equivalent here.** ResNet50V2 was better when frozen (0.9687 versus 0.9457) and marginally better at the end (0.9951 versus 0.9943), but both finished with 75 errors in the rerun, and the frozen-stage gap closing suggests that the data, not the architecture, sets the ceiling. **Flip test-time augmentation gave no benefit** (0.9943 with and without it) and was dropped. **Ensembling was consistently best on validation**, cutting errors by 27% in run 1 (46 versus 63) and 29% in the rerun (53 versus 75) relative to the better single model, and averaging four baseline models cut them to 43. **Weight decay with label smoothing helped on validation but not demonstrably on the leaderboard**: EfficientNetB0 errors fell from 73 and 75 to 60 and 60 and the four-model ensemble from 43 to 35, but ResNet50V2 was mixed, v4a scored 0.9784 against 0.9790 for v3, and the six-model mix with 45 validation errors scored 0.9800. **Further backbones did not help.** EfficientNetV2-S and ConvNeXt-Tiny, trained with the same recipe, were weaker single models (129 and 97 errors against 59 to 75), and no ensemble that included them had fewer than 35 errors, although the exploratory ten-model average scored 0.9804 on the leaderboard. Pretrained weights, partial unfreezing and learning-rate scheduling are supported by this evidence; augmentation, the resize and dropout were not ablated, so their effects are assumed, and weight decay and label smoothing were changed together.

#	Model / setup	Val. acc.	Val. errors	Kaggle
1	EfficientNetB0 stage 1 (frozen, lr 1e-3, 5 epochs)	0.9457	–	–
2	EfficientNetB0 stage 2 (top blocks unfrozen, lr 1e-4, scheduler)	0.9943	73	0.9767 (v1)
3	ResNet50V2 stage 1 (frozen)	0.9687	–	–
4	ResNet50V2 stage 2 (conv5 unfrozen; early stop, epoch-8 weights)	0.9951	63	not submitted
5	EfficientNetB0 + horizontal-flip test-time augmentation	0.9943	73	not submitted
6	Ensemble: average of #2 and #4	0.9964	46	0.9786 (v2)
7–9	Clean rerun of #2, #4, #6 (identical code, seed 42)	0.9941 / 0.9941 / 0.9958	75 / 75 / 53	not submitted
10	Equal-weight average of 4 baseline models (#2, #4, #7, #8)	0.9966	43	0.9790 (v3)
11	EfficientNetB0 stage 2 with AdamW (weight decay 0.05) + label smoothing 0.1, seed 42	0.9953	60	not submitted
12	Same as #11, seed 7	0.9953	60	not submitted
13	Average of #11 and #12 (baseline pair #2 + #7: 66 errors)	0.9955	57	not submitted
14	Equal-weight average of 6 models (#2, #4, #7, #8, #11, #12)	0.9965	45	0.9800 (v4b; outside pre-set rule)
15	ResNet50V2 stage 2, AdamW + label smoothing, seed 42	0.9950	64	not submitted
16	Same as #15, seed 7	0.9954	59	not submitted

#	Model / setup	Val. acc.	Val. errors	Kaggle
17	Average of #15 and #16 (baseline pair #4 + #8: 59 errors)	0.9957	55	not submitted
18	Equal-weight average of 4 regularised models (#11, #12, #15, #16)	0.9973	35	0.9784 (v4a)
19	EfficientNetV2-S, same two-stage recipe (last 20% of layers unfrozen, seed 42)	0.9899	129	not submitted
20	ConvNeXt-Tiny, same recipe	0.9924	97	not submitted
21	Equal-weight average of 10 models (#2, #4, #7, #8, #11, #12, #15, #16, #19, #20)	0.9971	37	0.9804 (v5; exploratory, outside pre-set rule)

Table 13: Experiment log (validation set; Kaggle public score where submitted).

The clean rerun, with identical code and seeds, shows the limits of a single run. Validation errors moved by about ten for the single models (EfficientNetB0 73 to 75, ResNet50V2 63 to 75), and the rerun changed 331 and 233 of 29,443 test predictions for v1 and v2 (about 1.1% and 0.8%). Differences of around ten errors are within run-to-run noise, and the backbone ranking flips between runs. The v1-to-v2 leaderboard gain (+0.0019) is about the size of the effect of simply retraining, so the leaderboard alone cannot separate the ensemble's contribution from training variability, even though validation shows it helping in both runs. Only the EfficientNetB0 improvement from regularisation (13 to 15 fewer errors, in both seeds) narrowly exceeds this noise; the ResNet50V2 changes are not. Figure 38 shows the experiment log as written by the notebook.

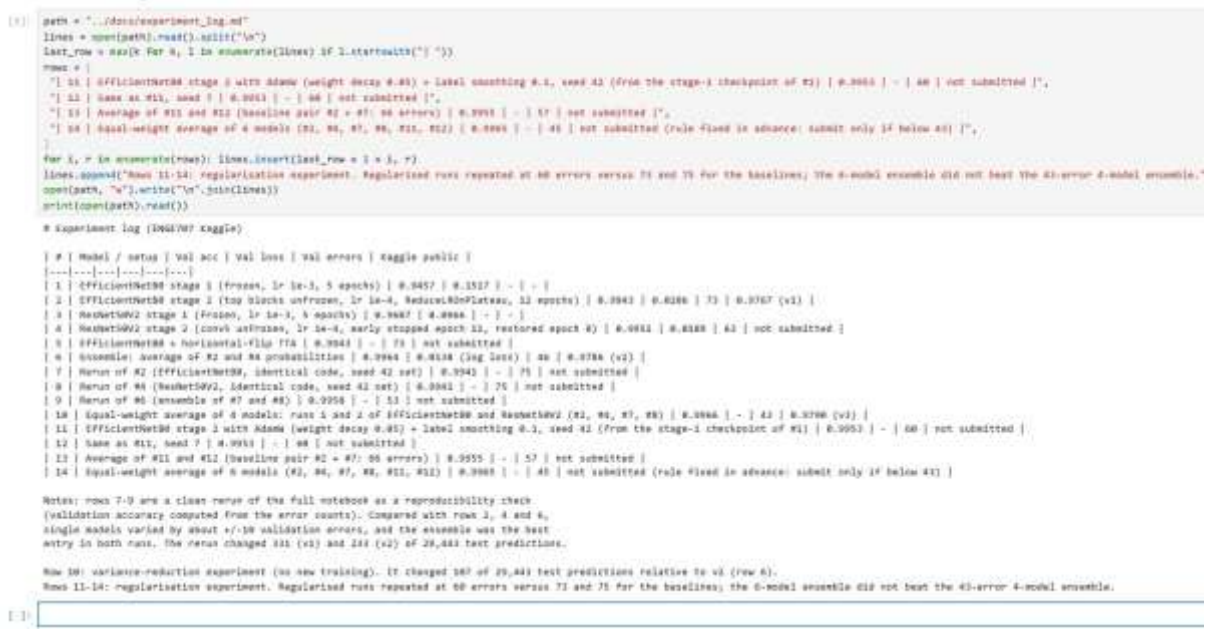


Figure 38: Experiment log after the regularisation experiment (rows 11 to 14 as logged by the notebook; rows 15 to 21 and the submission scores were added afterwards).

6.4 Possible causes of the validation–leaderboard gap

Hash-detected duplicates do not explain the gap. Only 143 validation images (1.12%) have a duplicate or near-duplicate in the training subset, and re-scoring the saved run-1 predictions without them (Figure 39) shows that none of the 143 was misclassified by EfficientNetB0, ResNet50V2 or their ensemble. Removing them therefore removes no errors, and accuracy on the remaining 12,589 images is 0.9942, 0.9950 and 0.9963 against 0.9943, 0.9951 and 0.9964 on all 12,732, a change of 0.01 points or less against a gap of about 1.8 points. That every flagged image is correct is consistent with the models having seen near-copies in training. This controls only for hash-detected copies and only for the run-1 models. Two

candidate explanations remain, neither tested. (1) Same-photograph leakage: the class counts suggest each photograph yields five crops (one face, two eyes, one nose, one mouth), so the 63,660 images would come from about 12,732 photographs, and a random image-level split can put one eye in training and the other in validation. Such crops are not hash duplicates but still make validation easier; a group-aware split would test this. (2) Harder test data: the test set may contain more small or blurry crops, which matters because errors rise steeply for small images (Table 12); comparing test and validation image sizes and blur would test this, but the competition rules restrict the test images to prediction, so it was not done. Both fit the lower test confidence. Until one is tested, the leaderboard score is the better estimate of generalization.

```
[4]: y = pipeline.val_df["label"].values  # true validation classes
mask = np.array([f in flag for f in pipeline.val_df["filename"].values])  # True for every flagged validation image
P = {"EfficientNetB0": np.load("../models/val_probs.npy"), "ResNet50V2": np.load("../models/val_probs_r50.npy")}  # saved validation probabilities (run 1)
P["Ensemble (v2)"] = (P["EfficientNetB0"] + P["ResNet50V2"]) / 2  # equal-weight average of the two models
for k, p in P.items():  # evaluate each model in turn
    err = p.argmax(1) != y  # True where the prediction is wrong
    print(f"{k:15s} all: {err.sum():3d} errors ({1-err.mean():.4f}) | flagged {mask.sum()}: {err[mask].sum()} errors | "  # errors on all images, then on the flagged ones
          f"clean {(~mask).sum()}: {err[~mask].sum()} errors, accuracy {1-err[~mask].mean():.4f}")  # errors and accuracy on the clean (unflagged) images

EfficientNetB0  all:  73 errors (0.9943) | flagged 143: 0 errors | clean 12589: 73 errors, accuracy 0.9942
ResNet50V2      all:  63 errors (0.9951) | flagged 143: 0 errors | clean 12589: 63 errors, accuracy 0.9950
Ensemble (v2)   all:  46 errors (0.9964) | flagged 143: 0 errors | clean 12589: 46 errors, accuracy 0.9963
```

Figure 39: Leakage-controlled validation (notebook 05): errors on all 12,732 validation images, on the 143 flagged images (no errors for any model) and on the 12,589 remaining images.

6.5 Improvement strategies

The proposals in Table 14 are ordered by expected value for effort, and **none has been tested**. A weighted loss for the Eye classes is deliberately not proposed, since they are already the best-performing classes.

Proposal	Expected benefit	Rationale
Group-aware validation split (cluster image embeddings to approximate source photograph, then split by cluster)	More honest validation; better model selection	Tests the same-photograph candidate in Section 6.4; if it is the cause, validation accuracy should fall towards the leaderboard score
Find and remove or relabel junk crops (for example, review the highest-loss training images)	Fewer confident errors; better calibration	Figure 37 shows non-informative crops and ambiguous labels (Northcutt et al., 2021); label smoothing helped on validation (Section 4.5), but cleaning the labels addresses the cause
Aspect-ratio-preserving resize with padding	Less distortion of thin or wide crops	Current resizing stretches every crop to a square, which distorts the shape of mouth and face crops
Higher input resolution or better upsampling for the smallest crops	Possible gains on the ≤64-pixel buckets	These buckets hold 68% of errors, though native detail is limited
More ensemble members or seeds, then a final model trained on train plus validation data	Further variance reduction; more training data	Ensembling was the most consistent gain; rerun variation shows individual models are noisy
Longer stage-2 schedule or unfreezing more layers	Possibly lower loss	EfficientNetB0 hit the 12-epoch limit without early stopping, but the flat validation loss suggests limited headroom
Separate ablations of weight decay, label smoothing, dropout and augmentation	Shows which regularisers actually help	Weight decay and label smoothing were changed together (Section 4.5), and dropout and augmentation were never ablated

Table 14: Proposed improvements (all untested).

6.6 Strengths and weaknesses of the final solution

Strengths. One shared, verified pipeline feeds all splits; submissions were validated against the required format; training behaviour was explained rather than only reported; the ensemble gain was measured in two runs; submissions

were governed by rules fixed in advance, with the two exceptions (v4b and v5) labelled as exploratory; and measured results are kept separate from hypotheses.

Weaknesses. No custom-CNN or ViT baseline; no ablation of dropout, augmentation or the resize, and weight decay and label smoothing were tested only together; validation possibly optimistic (same-photograph leakage is untested, although hash-detected duplicates were shown to matter little, Section 6.4), so validation gains such as the 35-error ensemble did not carry over to the leaderboard; overlap between labelled and test images not checked; run-to-run variation of about ten errors, so public-score differences of 0.0020 or less among the ensembles are within retraining noise (Section 6.3); and noisy crops that cap achievable accuracy.

Task 7: Conclusion and Reproducibility

A two-stage transfer-learning pipeline combining EfficientNetB0 and ResNet50V2 by probability averaging reached 99.64% validation accuracy and a public Kaggle score of 0.9786. Adding weight decay and label smoothing improved the best validation result to 99.73% (35 errors, four regularised models), while the best public scores (0.9800 and 0.9804) came from ensembles with 45 and 37 validation errors, so validation and the leaderboard did not rank the ensembles the same way. Fine-tuning gave the largest gain, learning-rate scheduling and early stopping kept overfitting mild, and ensembling cut validation errors by 27–29% in two runs, whereas two further backbones (EfficientNetV2-S and ConvNeXt-Tiny) did not reduce them further. The remaining errors cluster in cat nose/mouth confusion, tiny images and non-informative crops, so data quality, not model capacity, now limits performance. The validation-to-leaderboard gap of 1.7–1.9 points shows that validation overstates generalisation, with two untested candidate explanations (same-photograph leakage and harder test crops); hash-detected duplicates explain 0.01 points or less. The most valuable next steps are a group-aware validation split and a cleaner training set.

Code. The GitHub repository (https://github.com/Suemon-Kwok/Kaggle-Competition-Facial-Feature-Classification-Challenge) contains the notebooks (01_eda.ipynb, 02_preprocessing.ipynb, 03_model.ipynb for run 1, 03_model_rerun.ipynb for the clean rerun, 04_regularisation_experiment.ipynb, 05_leakage_controlled_validation.ipynb and 06_ensemble_extension.ipynb), src/pipeline.py, requirements.txt, docs/experiment_log.md, the figures, and the submission files (submission_v1_effnetb0.csv, submission_v2_ensemble.csv, submission_v3_ensemble4.csv, submission_v4_reg_ensemble4.csv, submission_v4_ensemble6.csv and submission_v5_exploratory_all10.csv). To reproduce: (1) create a Python 3.12 environment and install requirements.txt; (2) place the competition data in data/ (train/, test/, train_labels.csv, sample_submission.csv); (3) run the notebooks from the notebooks/ folder in the order given in the repository README (03_model steps 1-25, then 03_model_rerun, then 03_model steps 26-29, then 04 to 06); (4) the submission CSVs are written to submissions/ and models, logs and figures to models/ and figures/. Kaggle name: Suemon Kwok; leaderboard evidence is in Figures 34 to 36. Code comments were added to the repository after the figures were captured, so the notebook screenshots in this report show uncommented code; the committed notebooks and src/pipeline.py are commented.

Academic integrity. All code, training runs and analysis were carried out for this individual assessment using only the competition dataset. External libraries, pretrained weights and papers are cited in the reference list. A.I. assistance was used code drafting and sound boarding ideas.

Github repository

https://github.com/Suemon-Kwok/Kaggle-Competition-Facial-Feature-Classification-Challenge

Github repository commit history

https://github.com/Suemon-Kwok/Kaggle-Competition-Facial-Feature-Classification-Challenge/commits/main/

References

Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., … Zheng, X. (2016). TensorFlow: A system for large-scale machine learning. *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16)*, 265–283.

Chollet, F., et al. (2015). *Keras* [Computer software]. https://keras.io

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 248–255.

Dietterich, T. G. (2000). Ensemble methods in machine learning. In *Multiple Classifier Systems (MCS 2000)*, Lecture Notes in Computer Science, vol. 1857 (pp. 1–15). Springer.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., … Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. *International Conference on Learning Representations (ICLR 2021)*.

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*, 357–362.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Identity mappings in deep residual networks. In *Computer Vision – ECCV 2016*, Lecture Notes in Computer Science, vol. 9908 (pp. 630–645). Springer.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95.

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, PMLR 37, 448–456.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR 2015)*.

Kornblith, S., Shlens, J., & Le, Q. V. (2019). Do better ImageNet models transfer better? *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2661–2671.

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *International Conference on Learning Representations (ICLR 2019)*.

McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61.

Northcutt, C. G., Athalye, A., & Mueller, J. (2021). Pervasive label errors in test sets destabilize machine learning benchmarks. *35th Conference on Neural Information Processing Systems (NeurIPS 2021), Datasets and Benchmarks Track*.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research, 15*(56), 1929–1958.

Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). Rethinking the Inception architecture for computer vision. *2016 IEEE Conference on Computer Vision and Pattern Recognition*, 2818–2826.

Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. *Proceedings of the 36th International Conference on Machine Learning (ICML)*, PMLR 97, 6105–6114.

Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014). How transferable are features in deep neural networks? *Advances in Neural Information Processing Systems 27 (NIPS 2014)*, 3320–3328.